

NLP-BASED TOXIC COMMENT CLASSIFICATION WITH EXPLAINABILITY

Mini NLP Project Report

PREPARED BY: POOJA PRADEEP

A text classification system for identifying Toxic and Clean online comments using Natural Language Processing, TF-IDF, Logistic Regression and LIME.

1. Problem Statement

Online platforms receive a very large number of comments every day. Among these comments, some may contain insults, harassment or other forms of toxic language. Manually checking every comment is difficult when the volume of content is high. Therefore, an automated text classification system can be useful for identifying potentially toxic comments and helping moderators review them more efficiently.

The aim of this project is to develop a simple NLP-based system that classifies an online comment as either Clean or Toxic. In addition to classification, the project applies LIME (Local Interpretable Model-agnostic Explanations) to selected predictions so that the words that contributed to an individual prediction can be examined.

2. Objectives

- To classify online comments into Toxic and Clean categories.
- To preprocess raw text and prepare it for machine-learning based classification.
- To convert text into numerical features using TF-IDF.
- To train a practical and understandable Logistic Regression classifier.
- To evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
- To use LIME to provide word-level explanations for individual predictions.
- To identify the limitations of the system and suggest possible improvements.

3. Dataset

The project uses the Civil Comments dataset. The dataset contains comments collected from independent news sites and provides a toxicity score between 0 and 1. The dataset was loaded directly through Hugging Face using the dataset identifier `google/civil_comments`.

- Working sample: 30,000 comments.
- Original text attribute: comment text.
- Toxicity attribute: a continuous toxicity score between 0 and 1.
- Binary target: toxicity ≥ 0.5 was labelled Toxic, while toxicity < 0.5 was labelled Clean.
- Class distribution: 27,655 Clean comments (92.18%) and 2,345 Toxic comments (7.82%).

Only a 30,000-comment sample was used to keep the experiment practical in Google Colab. The class distribution is highly imbalanced, with Clean comments forming the majority. This is important when interpreting the results because a high accuracy can otherwise be misleading if the model mainly predicts the majority class.

4. Methodology

The project follows a complete NLP classification workflow, starting from raw comments and ending with both model predictions and explanations. The main stages are dataset loading, label preparation, text preprocessing, feature extraction, model training, evaluation and explainability.

- Load the Civil Comments dataset and select a reproducible sample of 30,000 comments.
- Convert the continuous toxicity score into the two target classes, Clean and Toxic.
- Clean the text using standard NLP preprocessing techniques.
- Split the data into 80% training data and 20% testing data using stratification.
- Represent the cleaned comments as 5,000 TF-IDF features.
- Train Logistic Regression with balanced class weights.
- Evaluate predictions on the unseen test set using multiple classification metrics.
- Apply LIME to selected comments to understand the words influencing individual predictions.

5. Implementation

The project was implemented in Python using Google Colab. The implementation was designed to keep the workflow straightforward and easy to understand while still covering the important steps of an NLP classification project.

5.1 Data Loading and Label Preparation

The Civil Comments dataset was loaded using the Hugging Face datasets library and converted into a pandas DataFrame. A sample of 30,000 comments was selected using `random_state=42` so that the experiment could be reproduced. The original toxicity value is continuous, so it was converted into a binary target for this project: a toxicity score of 0.5 or above is treated as Toxic, and a score below 0.5 is treated as Clean.

5.2 Text Preprocessing

Raw comments cannot be directly given to a traditional machine-learning classifier, so the text was cleaned before feature extraction. The preprocessing function first converts the text to lowercase so that words with different capitalization are treated consistently. URLs and line breaks are removed, followed by punctuation and numeric characters. The text is then split into individual words, common English stopwords are removed, and very short tokens are excluded.

This preprocessing reduces unnecessary variation in the text and leaves a cleaner set of words for feature extraction. The aim is not to remove meaningful language, but to reduce noise that may not contribute much to the classification task.

5.3 Train-Test Split

After preprocessing, the dataset was divided into 80% training data and 20% testing data. The split was performed using `random_state=42` for reproducibility and stratification was used to preserve the proportion of Clean and Toxic comments in both sets. This resulted in 24,000 training comments and 6,000 test comments.

5.4 TF-IDF Feature Extraction

Machine-learning models require numerical input, so the cleaned comments were converted into numerical features using TF-IDF (Term Frequency-Inverse Document Frequency). A TF-IDF score reflects how important a word is to a particular document while reducing the importance of words that occur frequently across many documents.

TF-IDF was selected because it is a simple and effective representation for text classification. It works well with sparse, high-dimensional text data and allows the Logistic Regression model to learn which words or word patterns are associated with the Toxic and Clean classes. In this implementation, the vectorizer was limited to the 5,000 most useful features to keep the model practical and computationally manageable.

5.5 Logistic Regression Model

The classification model used in the project is Logistic Regression. It was selected because it is a well-established method for binary classification and works effectively with high-dimensional sparse features such as TF-IDF vectors. It is also relatively fast to train and easier to understand and explain in a mini-project than a complex deep-learning model.

The dataset contains far fewer Toxic comments than Clean comments. To reduce the tendency of the classifier to favour the majority class, `class_weight='balanced'` was used. This gives more importance to the minority Toxic class during training. The model was trained with `max_iter=1000` to provide enough iterations for convergence.

5.6 Model Evaluation

After training, the model was evaluated using the 6,000 comments that were kept separate as test data. Accuracy, precision, recall and F1-score were considered together rather than relying only on accuracy. A confusion matrix was also generated to see how many Clean and Toxic comments were classified correctly or incorrectly.

5.7 Explainability Using LIME

LIME was applied after model training to make individual predictions easier to interpret. For a selected comment, LIME examines the effect of changing or removing parts of the text and creates a local explanation of the prediction. The output highlights words that contribute towards the Toxic or Clean decision.

In the project, LIME was tested on examples representing both classes, including an insulting comment that was predicted as Toxic and a normal conversational comment that was predicted as Clean. This adds an explainability component to the project rather than presenting the classifier as a system that only produces a final label.

6. Results

The trained Logistic Regression model achieved an overall accuracy of 87% on the 6,000-comment test set. However, because the dataset is imbalanced, the Toxic-class results provide a more useful picture of the model's behaviour.

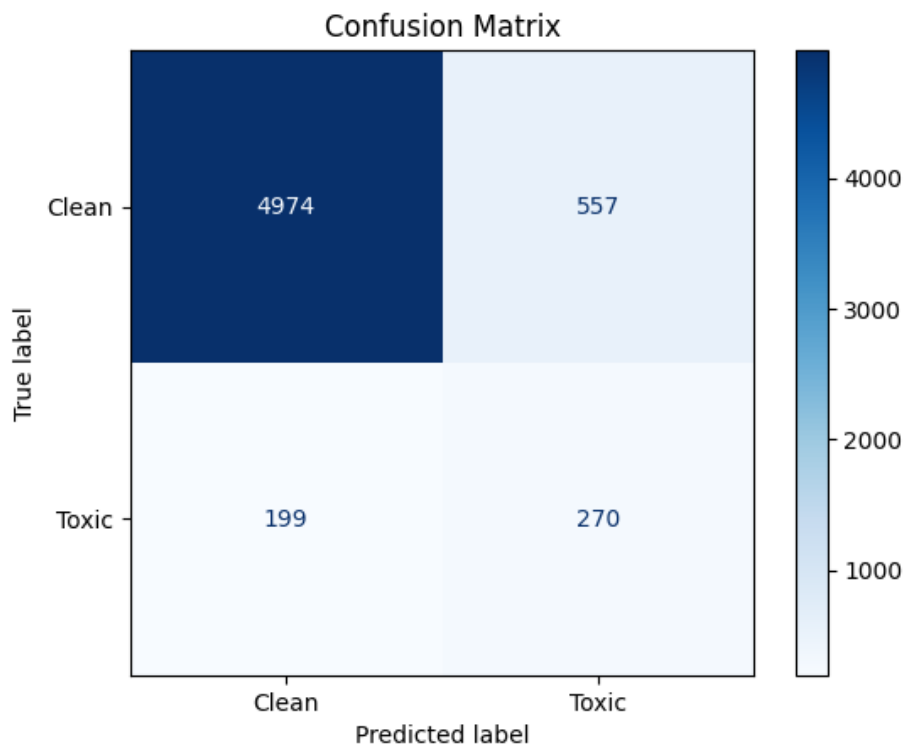
Class	Precision	Recall	F1-score	Support
Clean	0.96	0.90	0.93	5531
Toxic	0.33	0.58	0.42	469

The model correctly identifies a reasonable portion of the Toxic comments, with a Toxic recall of 0.58. This means that 58% of the genuinely toxic comments in the test set were detected. At the same time, the Toxic precision is 0.33, showing that a noticeable number of comments predicted as Toxic were actually Clean. Therefore, the model can be useful as a screening or flagging system, but it should not be treated as a final automatic moderation decision.

Classification report output:

...	precision	recall	f1-score	support
Clean	0.96	0.90	0.93	5531
Toxic	0.33	0.58	0.42	469
accuracy			0.87	6000
macro avg	0.64	0.74	0.67	6000
weighted avg	0.91	0.87	0.89	6000

Confusion matrix:

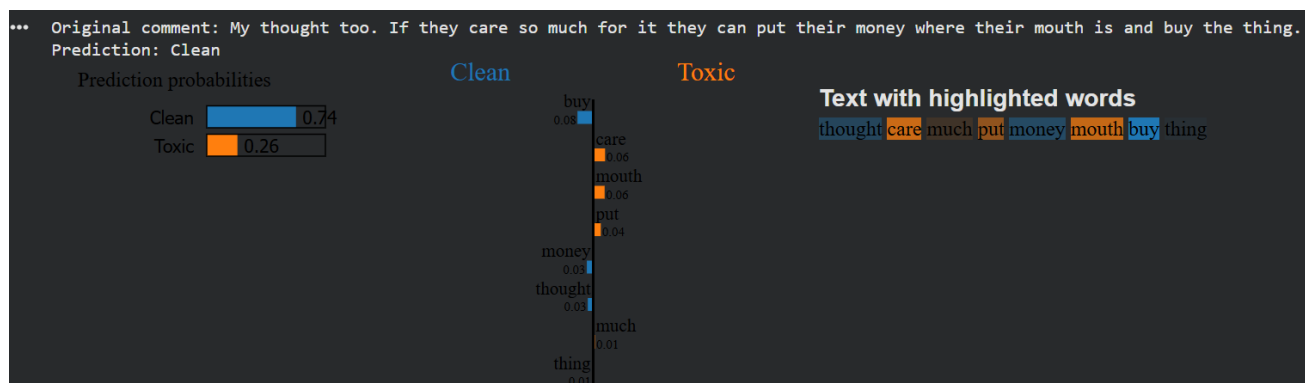


7. Explainability with LIME

LIME helps show why the classifier made a particular prediction for an individual comment. Instead of only displaying the class label, the explanation gives a local view of the words that influenced the prediction. This is especially useful in a moderation-related application because a human reviewer can inspect the reasons behind a model decision.

The project tested LIME with both Toxic and Clean examples. For example, the comment "You are so stupid, nobody wants to hear your opinion" was predicted as Toxic. A normal conversational example was predicted as Clean. The LIME output provides a word-level explanation for these individual decisions.

LIME output from the project:



8. Limitations

The project demonstrates a complete and understandable NLP workflow, but the current model has several limitations that should be considered when interpreting the results.

- Only 30,000 comments were used instead of the full dataset, so the model sees less language variety.
- The original toxicity score is continuous, but it was converted into a binary label using a 0.5 threshold.
- TF-IDF mainly represents word patterns and does not fully understand the context or meaning of a sentence.
- Sarcasm, indirect insults, coded language and context-dependent toxicity can be difficult to identify.
- The Toxic class is much smaller than the Clean class, which makes reliable toxic-comment detection more challenging.
- False positives and false negatives are possible, so the system is better suited to support human moderation than to replace it.

9. Future Scope

The current implementation can be improved in several ways. A larger training sample could provide greater language variety, while better feature engineering and model comparison could improve classification performance. More advanced language models could also be explored when greater contextual understanding is required.

- Train the model using a larger portion of the Civil Comments dataset.
- Experiment with n-grams and tuned TF-IDF settings.
- Compare Logistic Regression with Linear SVM using the same train/test split.
- Explore transformer-based models such as BERT or DistilBERT for better contextual understanding.

- Extend the system to support multiple languages.
- Add a confidence-based human review stage for uncertain predictions.
- Deploy the classifier as a simple web application or moderation API.

10. Conclusion

This project demonstrates a complete NLP-based approach for detecting toxic comments. The workflow includes data preparation, text preprocessing, TF-IDF feature extraction, Logistic Regression classification, performance evaluation and LIME-based explainability. The approach is intentionally simple enough to understand and explain while still addressing a practical real-world problem.

The results also show why accuracy should not be considered in isolation for an imbalanced classification problem. Although the model achieved 87% overall accuracy, the Toxic class achieved an F1-score of 0.42, indicating that there is considerable room for improvement. Overall, the project provides a solid foundation for toxic-comment detection and demonstrates not only model training, but also evaluation, explainability and critical analysis of the model's strengths and limitations.